Pininfarina
Istituto Tecnico Industriale Statale

Informatica e Telecomunicazioni

TrainTrack Manager

CANDIDATE
Cammarata Alessio

CLASS
5^A info

Academic Year 2023/2024

Declaration of Originality

I declare to be responsible for the content I'm presenting in order to obtain the final degree, not to have plagiarized in all or part of, the work produced by others and having cited original sources in consistent way with current plagiarism regulations and copyright. I am also aware that in case of false declaration, I could incur in law penalties and my admission to final exam could be denied

Abstract

This document provides a comprehensive explanation of the operation and development of the final project, TrainTrack Manager. TrainTrack Manager is an application designed to control a model railway layout using Arduino and a Python-based graphical user interface. The online documentation provides detailed instructions for installation and use, making the application accessible to all enthusiasts.

Contents

1 Considerations **1**

2 Introduction **1**

3 Project Objectives **2**

4 Methodology **2**

5 Project Overview and Clarifications **3**

5.1 Arduino Sub-project Operation 3

5.2 Graphical User Interface Operation 4

5.3 Automation System Operation 4

6 Arduino **5**

6.1 Arduino Physical Part 5

6.1.1 Arduino Mega 2560 5

6.1.2 Arduino Uno Rev 3 7

6.2 Arduino Logical Part 8

6.2.1 Arduino Mega 2560 8

6.2.2 Arduino Uno Rev 3 8

7 Graphical User Interface **10**

7.1 Motivation for Choosing Python 10

7.2 Program Structure 10

7.3 GUI Functionalities 11

7.4 GUI Design 11

7.4.1 Design Process 12

7.5 Technologies Used 12

7.6 Logical Dependencies Between Files 15

7.7 App Structure and Operation 16

7.7.1 Main Page 16

7.7.2 Settings Page 19

7.7.3 Locomotive Management Page 20

7.7.4 Circuit Monitoring Page 21

7.8 Documentation and Contributions 23

8 Automation System **24**
8.1 Hardware & Software Components 24
8.2 Control Algorithm 24
8.2.1 RFID Sensor Data Acquisition 24
8.2.2 Turnout Control Logic 25
8.2.3 Collision Prevention 25
8.3 Automatic Mode 26
8.3.1 Activating Automatic Mode 26
8.3.2 Automated Operations 27
8.4 Testing and Optimization 28
8.4.1 Testing Procedures 28
8.4.2 Optimization 28
8.5 Final Considerations 28
9 Future Developments **29**
9.1 Further Improvements 29
10 Gantt Diagram **30**
11 WBS **32**
12 Cost Tables **33**
References 35

1 Considerations

The project was born from the need to **control my father's model railway layout via computer**. As it is a highly scalable project, it **is divided into three main parts:**

- Arduino
- Graphical User Interface
- Automation System

Regarding the **timeline**, I decided to focus on the physical construction of the railway circuit until mid-January. The remaining time until the **deadline** (May-June) was primarily reserved for software development, although periodic checks on the layout were performed to ensure proper maintenance.

2 Introduction

The objective of the presented project is the **creation of a functional model railway system** that **includes multiple locomotives and their information management.** Furthermore, the construction of a **model railway layout** was planned, including a station and a train depot fully operational at maximum capacity. During the construction of the layout, bridges were also added to conceal the sensors used for the **automation system.**

The goal is to **apply the knowledge gained** during my studies to develop a **prototype application** capable of meeting the client's needs (my father).

3 Project Objectives

The project aims to:

- **Develop a comprehensive control system** for a model railway layout via computer.
- **Integrate Arduino** for the physical **control of locomotives and sensors.**
- Develop an intuitive graphical user interface for system management.
- **Implement an automation system** for the intelligent control of railway operations.
- **Ensure scalability and modularity** of the project for future expansions and improvements.

4 Methodology

To achieve the set objectives, the adopted methodology includes:

- **Requirements Analysis:** Collection and definition of the necessary hardware and software requirements.
- **Design**: Planning of the system architecture, divided into three main components (Arduino, Graphical User Interface, Automation System).
- **Development**: Implementation of physical components (Arduino) and software components (Graphical Interface and automation).
- **Testing and Verification:** Execution of system tests to verify the correct operation of all components.
- **Integration and Commissioning:** Merging the different components and testing the integrated system to ensure interoperability.
- **Documentation:** Creation of detailed project documentation, including user manuals and technical reports.

5 Project Overview and Clarifications

In this section, the various project sectors are defined in greater detail. As the project is composed of multiple components, the three sub-projects, their hardware requirements, and the necessary expertise are described individually.

5.1 Arduino Sub-project Operation

The Arduino-related part of the project is dedicated to creating a smart model railway layout controlled by Arduino. The materials required for the production of this model were:

- Arduino Mega 2560
- Arduino Uno Rev 3
- Motor Shield Rev 3
- Model Railway Layout on a wooden base
- 8 Step-by-step 18V Relays
- Pre-assembled 8-channel 5V opto-isolated relay board
- Miscellaneous wiring and cables
- 8 MFRC522 RFID readers and corresponding card tags
- Diodes and polarized LEDs

The expertise required for this part of the project is as follows:

- Proficiency in electronics and the creation of non-complex circuits
- Advanced knowledge of Arduino and the development of sensor-specific sketches
- Adequate knowledge of the model railroading world regarding the construction of layout buildings and tracks

5.2 Graphical User Interface Operation

The Graphical Interface portion of the project focuses on developing the locomotive control system. To achieve this, I decided to use Python's graphical capabilities. The necessary technical requirements are as follows:

- Proficiency in the Python programming language
- Adequate knowledge of various constructs and libraries:
  - Pyserial
  - Opencv
  - Os
  - Tkinter
  - Threading
  - Queue
  - Pillow
  - Subprocess
- Proficiency in Arduino communication methods

5.3 Automation System Operation

The objective of this part of the project is to create an algorithm for managing trains without manual control, thereby avoiding collisions. Circuit automation is ensured through the use of relays and RFID readers/tags. The latter are mounted on the locomotives; each must have its own tag. Through the interface, it is possible to register the locomotive tag in memory and use it freely within the circuit during automated mode. The readers are responsible for signaling the passage of a locomotive and are positioned at critical sites to provide the execution process with the best possible data for managing switches. The relays are tasked with alternating the position of the turnouts; they are automatically controlled by the algorithm, which decides whether or not to switch the turnout based on the received data.

6 Arduino

This part of the project has been divided for logistical reasons into **physical and logical parts.**

6.1 Arduino Physical Part

The **physical** part concerns **Arduino**, its use in the circuit, and the various **electrical circuits** created to allow the **management of turnouts and sensors**. Each Arduino has its own circuit and is used for different purposes.

6.1.1 Arduino Mega 2560

The Physical part related to Arduino Mega 2560 consists of:

- **Locomotives**
  Arduino Mega 2560 is the Arduino that **manages the modulated current** sent to the **decoders**; this current passes through the **Motor Shield Rev3** mounted on the Arduino, which allows for managing the **modulation** of current inputs through the DCCpp library (discussed in the next section). This allows **hiding messages within the current** that passes **through the tracks** to reach the **decoders** inside each locomotive, which **translate** the packets and **execute** the actions.

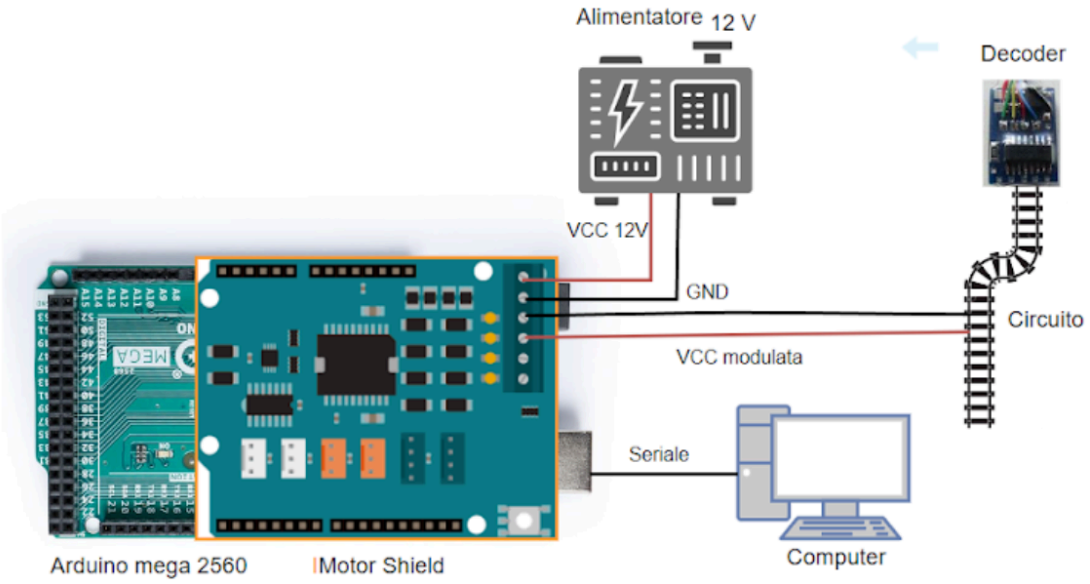


Figure 1: Wiring logic for locomotive control with Arduino Mega 2560

- **Turnouts**
  In addition to managing locomotives, Arduino Mega 2560 manages turnout activity. It is capable of sending 5V pulses from the digital pins to trigger the relays controlling the switches. These connections between the Arduino and relays require physical isolation; therefore, we used a pre-assembled 8-channel 5V opto-isolated relay board. This allows activating the relays via Arduino while isolating the two circuits to prevent interference. It was also necessary to isolate the relay circuit using diodes and polarized LEDs to absorb the back-EMF signals caused by the relay coils.

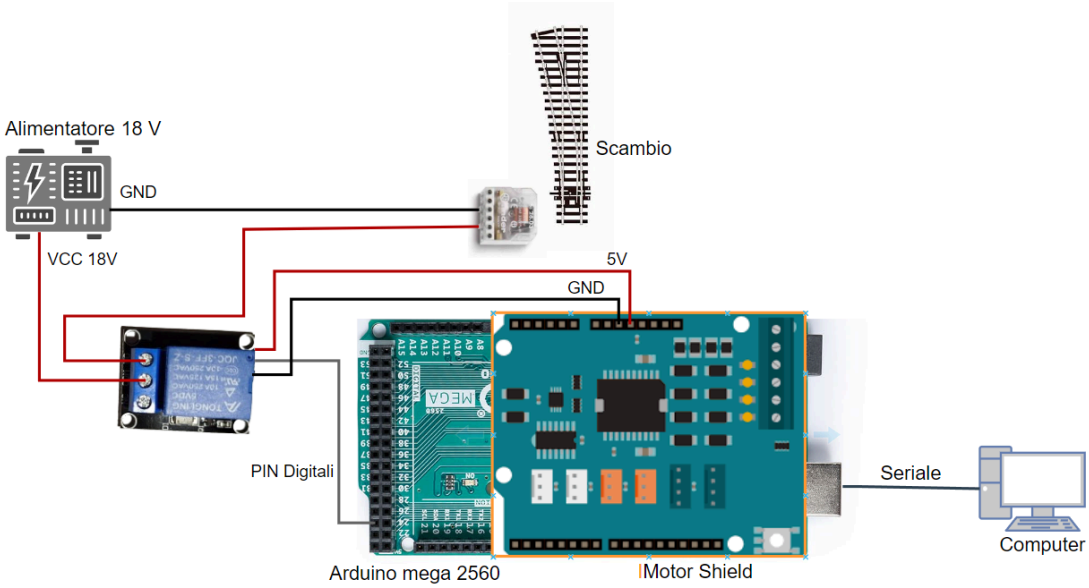


Figure 2: Wiring logic for turnout control with Arduino Mega 2560

6.1.2 Arduino Uno Rev 3

The connections for the **RFID sensors** are made with **Arduino UNO Rev 3**. It **manages all 8 sensors** on the layout and, upon receiving a message, sends it directly to the computer via serial communication, which then processes it in the algorithm. To implement the connections with Arduino, pins dedicated to **SPI communication** were used, along with 2 to 3 mm diameter cables to reduce voltage drops.

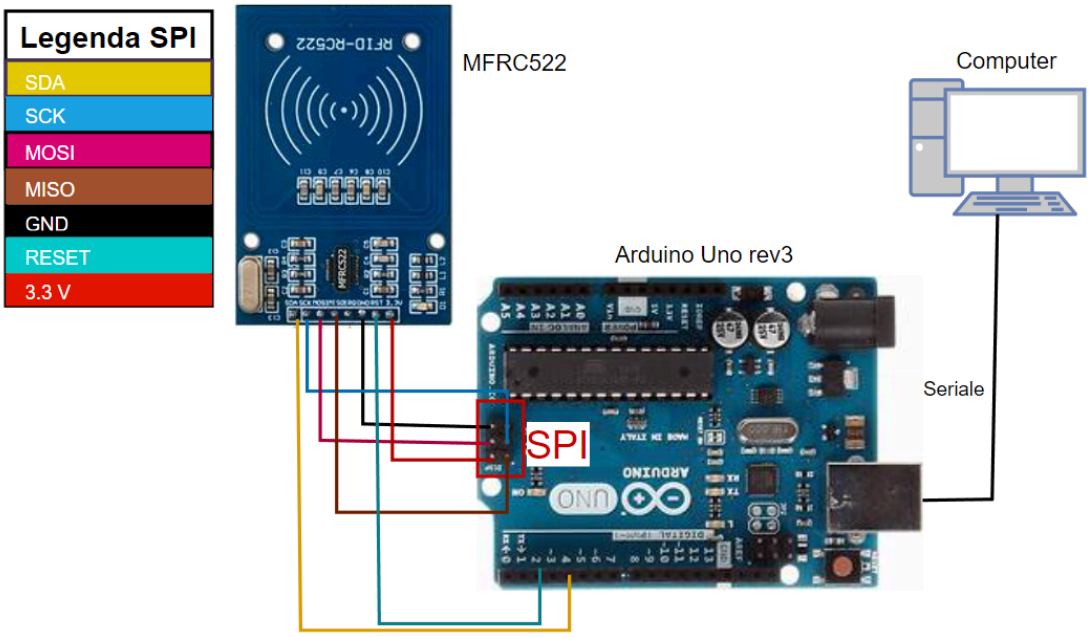


Figure 3: Wiring logic for MFRC522 sensors with Arduino Uno Rev 3

Technical documentation for the circuits is attached below.

- Technical documentation for the circuits [1]

6.2 Arduino Logical Part

The **logical** part includes the **C++ sketches** for the Arduinos, used both for moving the locomotives **(Arduino Mega 2560)** and recording their position **(Arduino UNO).**

6.2.1 Arduino Mega 2560

The sketch used for this Arduino is the **DCCpp library**, which handles locomotive management via **decoders**; this library is **Open Source** and easily implementable.
The process performed by this library is **re-modulating** the alternating current from the power supply to hide commands inside it. The decoder **translates and executes** these on the locomotives, allowing power and control to be sent via the tracks without the need for loose wires that would ruin the aesthetic.

Specifically, this library also defines the **memory management of information and the various commands** the Arduino must receive to perform specific actions. More details on the library are available in the attachments.

- DCCpp Library [2]

6.2.2 Arduino Uno Rev 3

The sketch used for this Arduino utilizes the **MFRC522 library by Miguel Balboa** and focuses on sensor management (also Open Source).
This code allows **managing sensor signals** and **sending personalized messages via serial** to the computer for processing. Notably, this sketch is **scalable and allows changing the number of sensors** as needed.
More details on the library are available in the attachments.

- MFRC522 Library [3]

Custom code example:

```
#include <SPI.h>
#include <MFRC522.h>

#define RST_PIN         2
#define NR_OF_READERS   8

byte ssPins[] = {4, 5, 6, 7, 8, 9, 10, 18};
MFRC522 mfrc522[NR_OF_READERS];   // Create MFRC522 instance.

void setup() {
    Serial.begin(115200); // Initialize serial communications with the PC
    while (!Serial);      // Do nothing if no serial port is opened
    SPI.begin();          // Init SPI bus

    // Init each MFRC522 card
    for (uint8_t reader = 0; reader < NR_OF_READERS; reader++) {
        mfrc522[reader].PCD_Init(ssPins[reader], RST_PIN);
        delay(100);
    }
}

void loop() {
    for (uint8_t reader = 0; reader < NR_OF_READERS; reader++) {
        // Look for new cards
        mfrc522[reader].PCD_Init();

        // Controls if a card is ready to connect
        if (mfrc522[reader].PICC_IsNewCardPresent() &&
mfrc522[reader].PICC_ReadCardSerial()) {
            // Print the specific message
            Serial.print(reader + 1);
            Serial.print("/");
            dump_byte_array(mfrc522[reader].uid.uidByte, mfrc522[reader].uid.size);
            Serial.println();
            Serial.flush();
            break; // Exit from the cycle to restart it faster
        }
    }
}

// Helper routine to dump a byte array as hex values to Serial.
void dump_byte_array(byte *buffer, byte bufferSize) {
    for (byte i = 0; i < bufferSize; i++) {
        Serial.print(buffer[i], HEX);
    }
}
```

7 Graphical User Interface

The goal of the graphical user interface (GUI) is to simplify the management of the locomotives. The project is open-source, and anyone can contribute to its improvement via the repository link:

- TrainTrack Manager [4]

7.1 Motivation for Choosing Python

The GUI for this project was developed in **Python**, a familiar language as it was part of the eleventh-grade curriculum. Furthermore, participation in several school-provided advanced courses over the years consolidated a preference for Python over other languages such as Java or C.

7.2 Program Structure

The GUI management program was developed using **Object-Oriented Programming (OOP)**. Specifically, there are **six different classes** that organize the GUI's functionality:

- **App:**
  This class is the main entry point of the program. It handles application initialization and coordinates the other classes.
- **GUI:**
  Manages the construction of the user interface, defining the layout and graphical elements such as windows, panels, labels, and input fields.
- **Buttons:**
  Contains the logic for creating and managing the interface buttons. Each button has a specific associated action.
- **Circuit_window:**
  Manages the window that displays the circuit status, showing the position of the locomotives and allowing manual control of the turnouts. It is also possible to start the automatic mode for a smoother and more interactive experience.

- **Algorithm:**
  Implements the algorithms necessary for the application's operation, such as locomotive control and sensor data processing.
- **Cam:**
  Manages camera-related functionalities, such as the real-time visualization of the layout.

7.3 GUI Functionalities

The main functionalities of the Graphical Interface are:

- **Train Management:**
  From the GUI, it is possible to store trains in memory, manage their presence in a table, and modify their data if necessary.
- **Locomotive Control:**
  Users can control trains, managing speed and direction with a simple click.
- **Position Visualization:**
  By opening the control window, it is possible to view the position of the trains on the tracks and manually command the turnouts.
- **Automatic Mode:**
  An automatic mode is available to manage the circuit autonomously, controlling turnouts through an algorithm and preventing collisions between locomotives. This mode supports a minimum of 2 locomotives and a maximum of 3.
- **Multilanguage Support:**
  Up to four languages are available:
  - Spanish
  - French
  - Italian
  - English

7.4 GUI Design

This section is the **core** of the project and required the **majority of the development time**. The **GUI is the operational center** that integrates all

system components, allowing for **intuitive and efficient interaction with the hardware.**

7.4.1 Design Process

The design process was guided by a series of well-defined steps:

1. **Requirements Definition:**
   Initially, I **identified the primary needs** of the interface, considering **key functionalities** such as train management, locomotive control, and position tracking.

   I created a **detailed list of necessary functionalities** and **desired features**, which served as a **guide** throughout the entire design and development process.

2. **Program Structure:**
   The GUI management program was divided into several classes, each playing a specific role in organizing and executing the interface functionalities.

3. **OOP Implementation:**
   All classes were implemented using the Object-Oriented Programming (OOP) paradigm, which promoted a **modular and easy-to-maintain structure.** This approach facilitated the **extension and maintenance of the code over time.**

4. **Testing and Optimization:**
   After completing the initial implementation, I performed a series of **tests** to ensure that all functionalities **operated correctly.**

   I **optimized the code** to improve performance and ensure the interface was **responsive and lag-free.** I also conducted **usability tests** to ensure the interface was **intuitive and easy to navigate.**

7.5 Technologies Used

As previously mentioned in the Clarifications section, these are the technologies used:

- **Tkinter:**
  Tkinter, Python's standard library for GUI creation, was used to build the graphical interface.

- **pySerial:**
  Communication with the Arduino UNO was managed via the **pySerial** library, which allows sending and receiving data through the serial port. This is necessary to read messages sent by the sensors during the algorithm's execution.

- **OpenCV:**
  An open-source library specialized in image and video processing, used in the `Cam` class to activate and display frames from the device's primary camera.

- **Os:**
  A Python library providing functionality for **interacting with the operating system**, highly useful for managing **file paths**, including those for **images and translation text files**. Furthermore, the use of this library allows the application to run on **macOS or Linux** systems.

- **Threading:**
  The **Threading** library in Python was used to handle concurrent programming, allowing multiple tasks to run simultaneously. Specifically, it was employed in the algorithm to **read sensor messages** and trigger actions based on those messages, while ensuring a **smooth and responsive visualization of GUI updates.** Thanks to this background processing capability, the program remains efficient and sensitive to user interactions.

- **Queue:**
  A library providing **data structures for queue management**, i.e., ordered sets of elements that allow insertion and extraction according to the "First-In, First-Out" (FIFO) principle. It is also used in the **algorithm** management to read messages in **FIFO** order and exchange read messages between different threads.

- **Pillow:**
  An **image processing library for Python** that allows **opening,**

manipulating, and saving images. In this project, it was primarily used for **resizing and rotating images** present in the circuit diagram, such as **sensors or buildings.**

- **Subprocess:**
  A library that **allows executing operating system commands** from a Python script and interacting with them. I preferred using `subprocess` over `os` (which performs similar functions) because it **does not require a terminal** during execution, allowing the same functions to be used **without visual disruptions for the user running the executable file.**

7.6 Logical Dependencies Between Files

Below is an image illustrating the **logical dependencies between the various program files and classes**, facilitating the **understanding of their interaction and overall structure:**

Figure 4: Logical structure of the TrainTrack Manager graphical interface

7.7 App Structure and Operation

The TrainTrack Manager graphical interface is composed of several pages, each designed to offer **specific functionalities and facilitate user interaction** with the railway system. First, being a desktop application, the desktop shortcut must be opened.

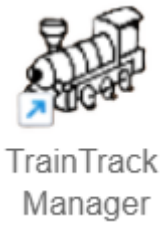


Figure 5: Desktop application images

The main pages of the interface are described individually below.

7.7.1 Main Page

The main page **is the entry point of the application**. Here, users find a summary of the **primary functionalities** and can **quickly access other pages**. It is also possible to **change the program language** via the dropdown menu that opens by clicking the flag in the top right corner.

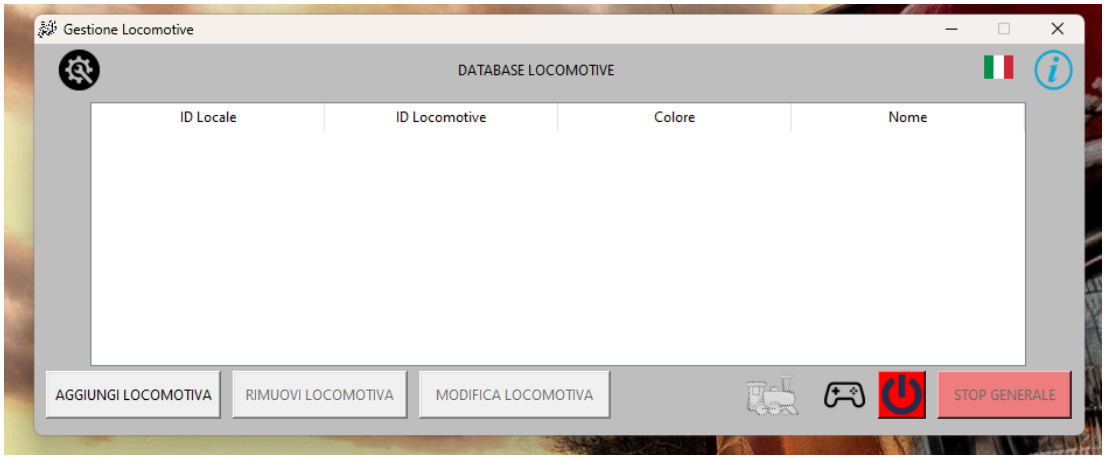


Figure 6: Main Page

The main page also serves the function of **locomotive management**; indeed, under the **"Locomotive Database"** heading, there is a **table** that populates as locomotive instances are added via the "Add" button. On the "Add Locomotive" page, **it is necessary to specify the locomotive name (which**

can be chosen freely) and the **LocoID of the decoder (which must be unique)** for each locomotive. If IDs match, **they can be modified via the "Edit" button.** Furthermore, **it is necessary to set a color**; if not specified, **one is assigned randomly from those available**, the number of which is strictly related to the maximum number of locomotives the system can handle.

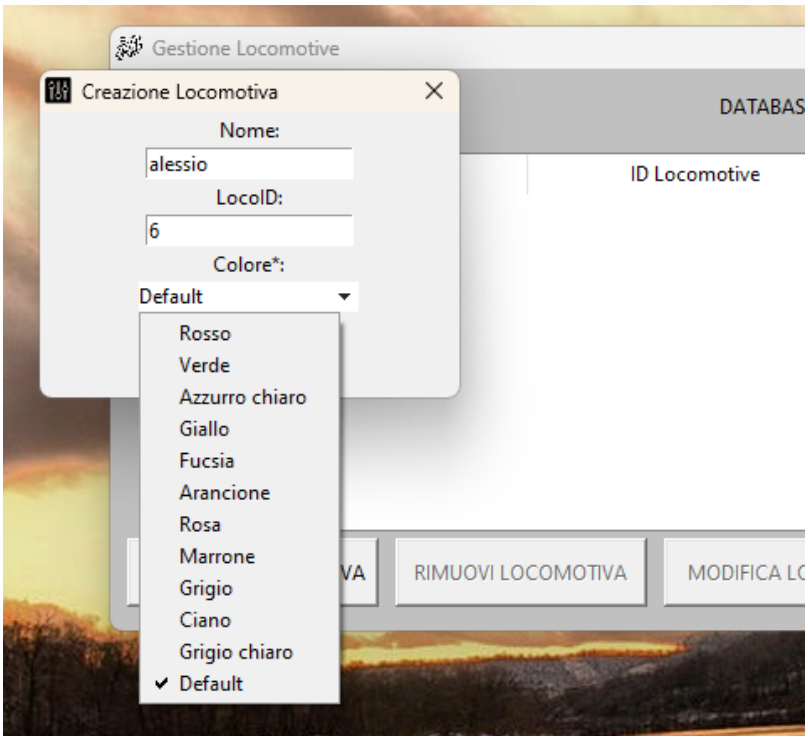


Figure 7: Locomotive addition/creation window

Saved train instances **can then be viewed in the table.**

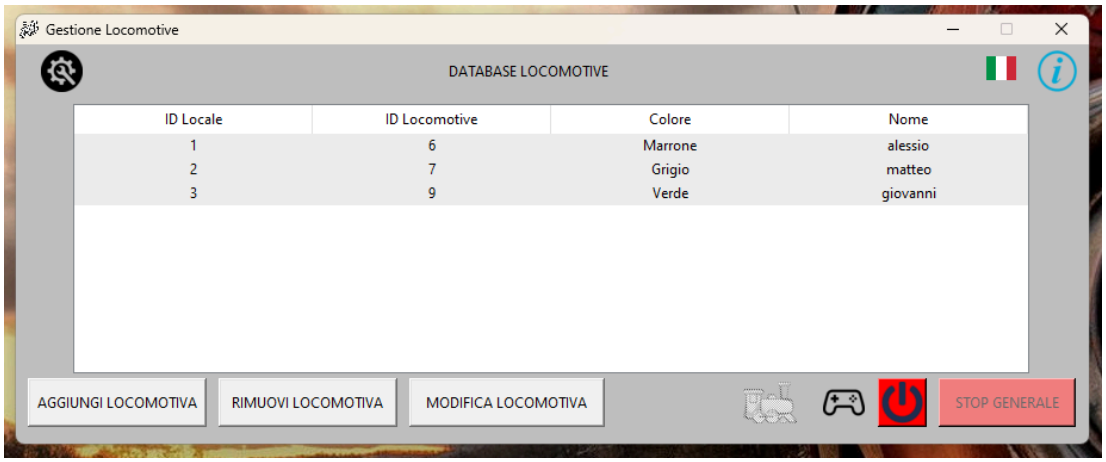


Figure 8: Main Page with table instances

They can then be **modified or removed using the relative buttons.**

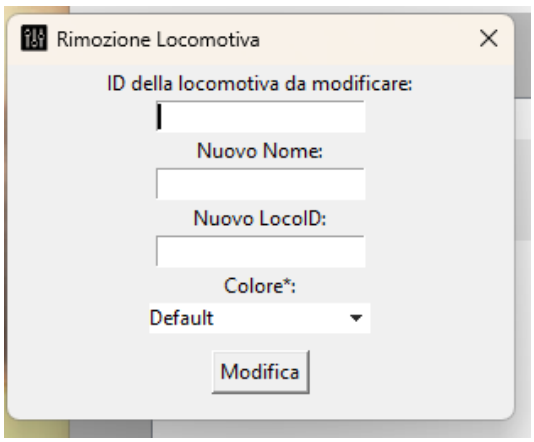


Figure 9: Locomotive modification window

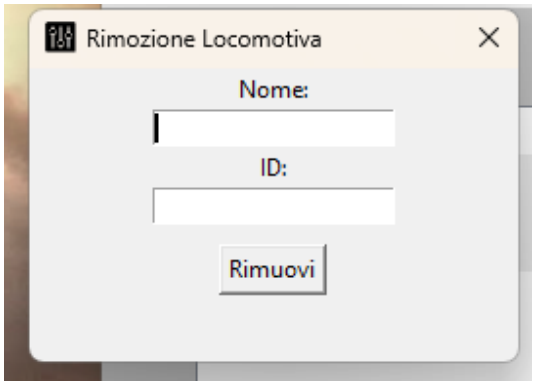


Figure 10: Locomotive removal window

It may be noted that some buttons **are disabled**; this is because the **power is off**. It is not possible to click buttons to control locomotives or stop the system if power is inactive. **For regular operation, it is therefore necessary to activate the power by pressing the 'o' key on the keyboard or using the power button, which will turn green once activated.**

Figure 11: Opening the power switch

At the same time, the other 2 buttons can be clicked **if and only if at least one locomotive is present in the table.**

7.7.2 Settings Page

The settings page allows users to **customize application preferences.** It is possible to **change the maximum number of locomotives (ranging from 1 to 11) and configure serial port settings**, such as which port to **associate with the sensors and which with the DCC base station**. It is also possible to disable ports if they are not needed. If devices are compatible, they will also be visible in the table.

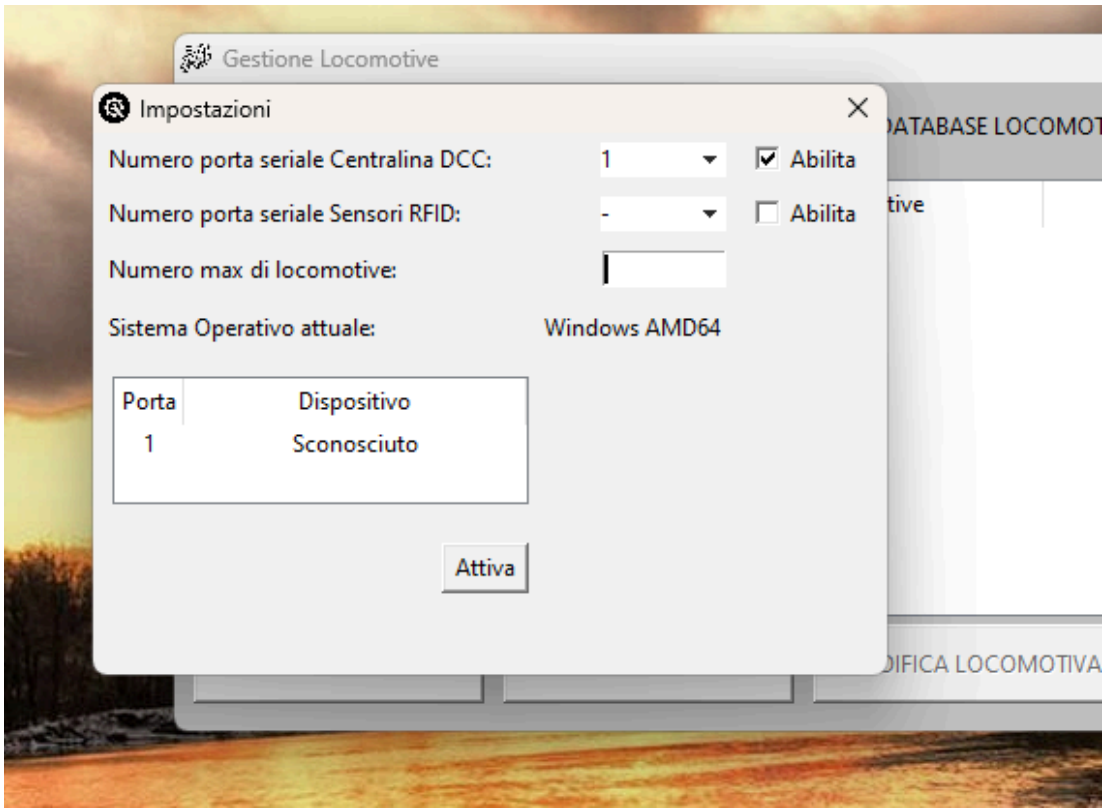


Figure 12: Settings window

7.7.3 Locomotive Management Page

This page allows users to manage the locomotives present in the system. It can be opened via the keyboard by pressing the locomotive's ID number, by clicking the locomotive in the table, or via the dropdown menu on the train icon—always remembering to have the power activated and at least one locomotive added to memory.

Figure 13: Dropdown menu for locomotives on the main page

It is possible to **assign their speed and direction**, and also to **stop the locomotive**. All of this happens in a personalized manner according to the locomotive's LocoID.

Figure 14: Control page for locomotive 'alessio'

7.7.4 Circuit Monitoring Page

The circuit **control and monitoring** window provides a graphical visualization of the railway layout. Users can **monitor locomotive positions and turnout statuses in real-time**, and it is also possible to **activate the camera.** Through this page, users can track the flow of railway traffic and **intervene** if necessary, **manually activating turnouts** to correctly route locomotives along the tracks.

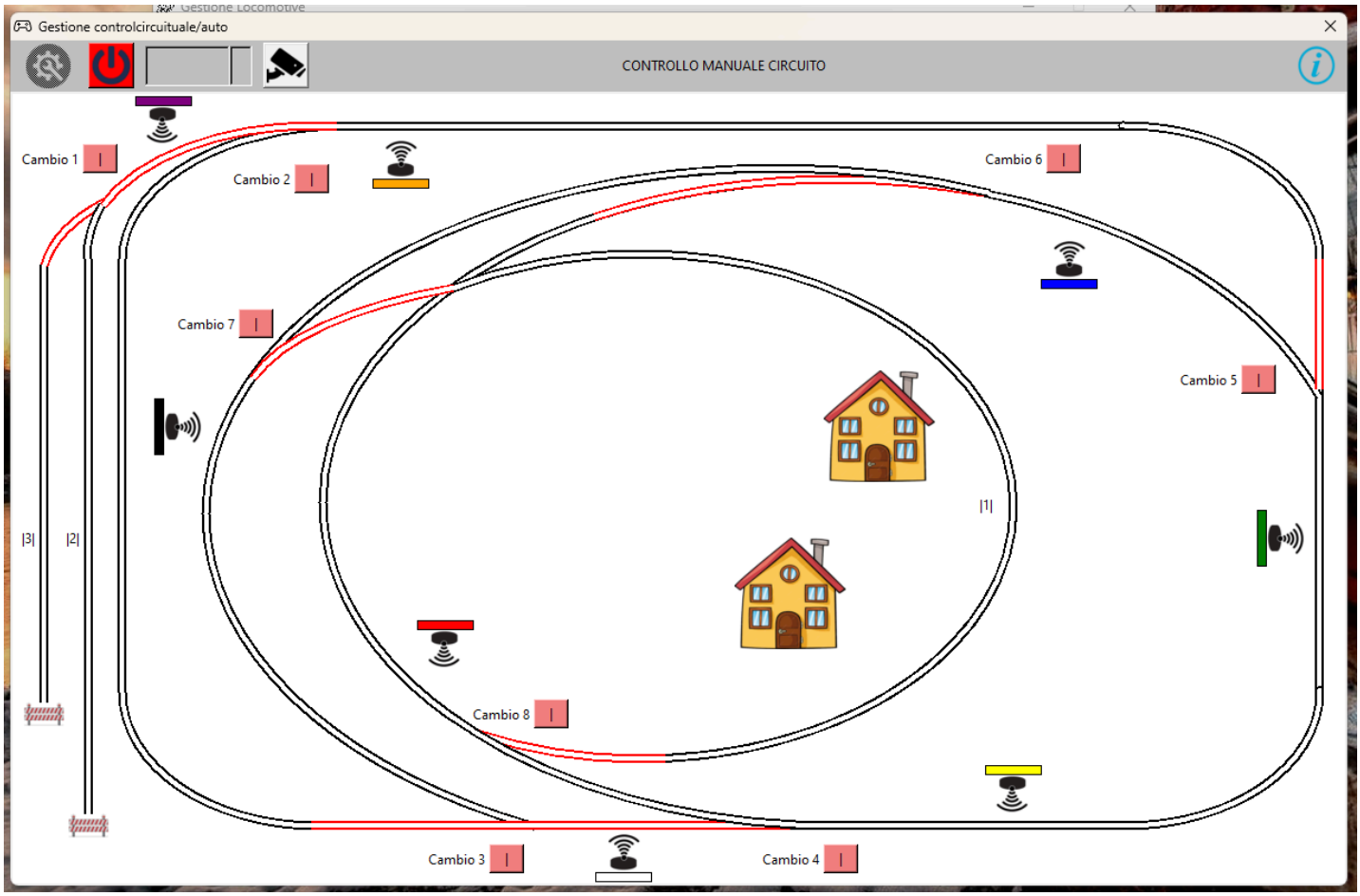


Figure 15: Circuit Monitoring Page

It is also possible to activate the **automatic mode** of the railway system. In this mode, the system autonomously controls railway operations using internal algorithms. Users can monitor system progress and intervene manually if necessary. The only **requirement** to access and activate it is to **first start the mode via the start button or the 'o' key on the keyboard**. In case of doubt, an **information section** is available **via the 'i' key on the keyboard** or by pressing the blue button.

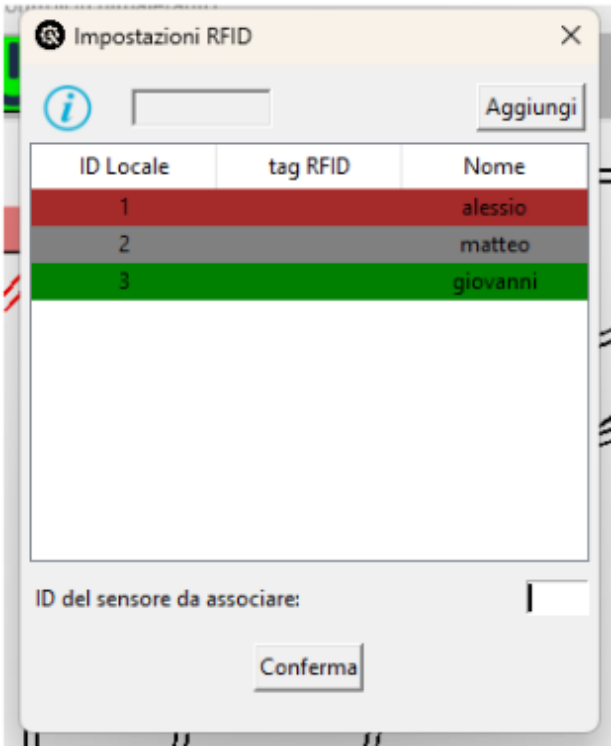


Figure 16: Circuit monitoring settings page

Afterward, it is necessary to **register the locomotives**. The procedure is simple: the **RFID tag of the locomotive must be passed under any sensor** on the circuit. The code will appear in the box located above the table, and it will be possible to **decide who to assign it to and then confirm.** In case of doubt, an **information section** is also available on this page via the 'i' key or by pressing the blue button. If one forgot to **add locomotives**, it can also be done from this page using the **Add button or by pressing '+' on the keyboard.**

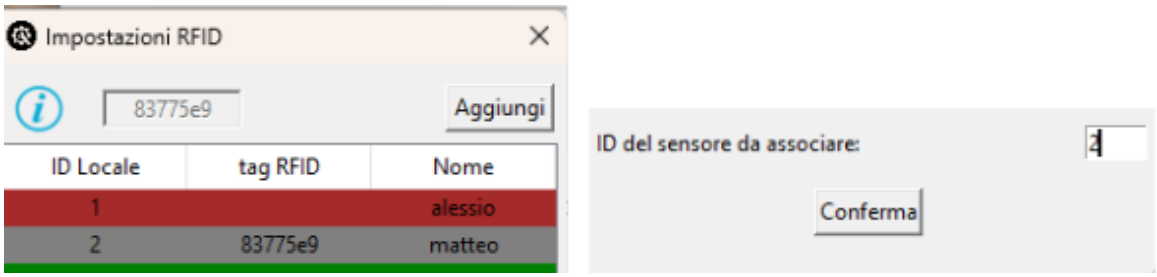


Figure 17: Inserting a tag on the circuit monitoring settings page

Returning to the circuit monitoring page, it is possible to **start**, as mentioned previously, **the camera view pointed at the circuit via the 'v' key on**

the keyboard or by pressing the button with a camera icon. The result is a window that can be resized according to the screen.

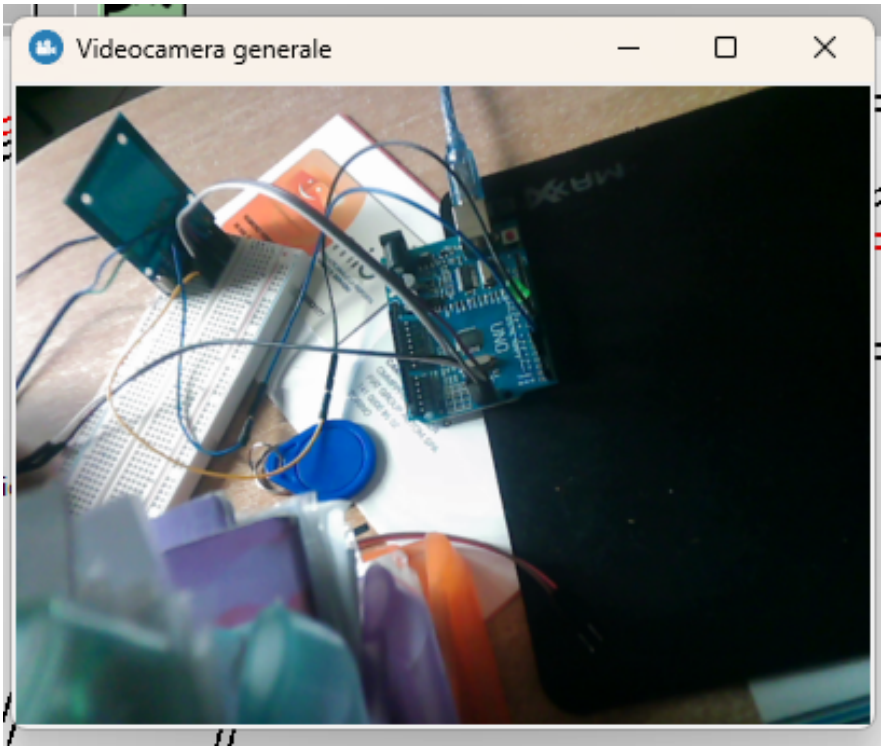


Figure 18: Camera page

7.8 Documentation and Contributions

Full **documentation** can be viewed at this link:

- GUI Technical Documentation [5]

8 Automation System

Model railway automation has been implemented using **relay and RFID technology**. This section describes how the hardware and software components work together to **automate the control of locomotives and turnouts, and how the control algorithm was designed.**
Currently, this part of the project is still **under development** and may exhibit some functional imperfections.

8.1 Hardware & Software Components

The **hardware and software components** are the same as those implemented in the manual system; the primary modification is the implementation of the **Control Algorithm**.

8.2 Control Algorithm

The control algorithm is the **heart of the automation system**, responsible for **processing information** from the sensors and **controlling the actuators.**
Currently, this algorithm is in the **optimization phase** and may not yet function perfectly.

8.2.1 RFID Sensor Data Acquisition

Data acquisition from RFID sensors operates through:

- **RFID Tag Reading:**
  The algorithms **continuously read data from detected RFID tags** along the circuit. Each reading contains information about the locomotive, from which speed, direction, and consequently position are derived.
- **Data Management:**
  Acquired data is stored in a FIFO (First-In, First-Out) structure using the **Queue library**, ensuring that **readings are processed in the correct order.**

8.2.2 Turnout Control Logic

The turnout control logic includes:

- **Direction Determination:**
  Based on the current position of the locomotives and the **intended destination**, the algorithms calculate the **correct direction for the turnouts.**
- **Executing the Switch:**
  Using relays, the algorithms send commands to **change turnout positions**, ensuring that locomotives follow the correct path.

8.2.3 Collision Prevention

Collision prevention includes:

- **Critical Point Detection:**
  When started, the algorithm **randomly assigns a route to each locomotive**, then **calculates the critical points**, which are simply the **intersections of the assigned routes.**
  In this way, only critical points are monitored while others can be ignored.
- **Safety Actions:**
  If a locomotive attempts to enter a point where another locomotive is present, the turnout will be activated and the **initial path will be diverted.**

Critical Point Calculation Code:

```
#Ritorna un vettore che contiene i percorsi liberi in quel momento
    def show_percorsi_liberi(self,id):
      return list(set(data.LRoutes.keys()) - set(data.percorsi_assegnati)) if
data.locomotives_data[id]['Direzione'] == 0 else list(set(data.RRoutes.keys()) -
set(data.percorsi_assegnati))

    #Ritorna l'intersezione tra i percorsi assegnati
    def trova_criticita(self):
        return set(data.LRoutes[data.percorsi_assegnati[0]]) &
set(data.RRoutes[data.percorsi_assegnati[-1]])

    def scegli_percorso(self, id):
        percorsi_liberi = self.show_percorsi_liberi(id)
        percorso_scelto = random.choice(percorsi_liberi)
        print("il percorso scelto è: " +str(percorso_scelto))
        data.percorsi_assegnati.append(percorso_scelto)

        data.locomotives_data[id]['Percorso'] = percorso_scelto
        if id ≠ 0:
            data.criticita = self.trova_criticita()
```

8.3 Automatic Mode

The automatic mode of the railway control system allows for **autonomous circuit management, improving the efficiency and safety of operations.**
Although in the optimization phase, the algorithm may still present some imperfections.

8.3.1 Activating Automatic Mode

Automatic mode activity includes:

- **Starting from the GUI:**
  Users can **activate automatic mode** with a simple click in the **graphical interface from the circuit window page.**
  Once activated, the system takes control of the locomotives and turnouts. Manual intervention to switch turnouts is still possible if necessary.

- **Real-Time Monitoring:**
  The GUI **displays the position of the locomotives and the status of the turnouts in real-time**, allowing users to monitor the circuit's progress.

Sensor and Turnout Data Code:

```
#Creazione in memoria dei deviatoi
Turnouts = {
    "Cambio 1":   [False,"","",""],
    "Cambio 2":   [False,"","",""],
    "Cambio 3":   [False,"","",""],
    "Cambio 4":   [False,"","",""],
    "Cambio 5":   [False,"","",""],
    "Cambio 6":   [False,"","",""],
    "Cambio 7":   [False,"","",""],
    "Cambio 8":   [False,"","",""]
}

#creazione in memoria dei sensori
Sensors = {
    "Sensore 1":   ["","","","","purple"],
    "Sensore 2":   ["","","","","orange"],
    "Sensore 3":   ["","","","","white"],
    "Sensore 4":   ["","","","","yellow"],
    "Sensore 5":   ["","","","","green"],
    "Sensore 6":   ["","","","","blue"],
    "Sensore 7":   ["","","","","black"],
    "Sensore 8":   ["","","","","red"]
}
```

8.3.2 Automated Operations

Automated operations consist of:

- **Locomotive Control:**
  The **speed and direction of the locomotives are managed automatically by the algorithms**, based on input data from the RFID sensors.
- **Turnout Management:**
  Turnouts are **opened or closed** autonomously, ensuring that **locomotives follow the planned route** without the need for manual intervention.
- **Accident Prevention:**
  Collision prevention algorithms operate **constantly**, ensuring **safe and uninterrupted circuit operation.**

Critical Point Control Code:

```
#   Questa funzione controlla che i percorsi adiacenti a quelli toccati dal treno,
nella direzione in cui sta andando siano liberi
    def control(self,Turnout,direzione,natural_link):

        for item in data.Adjacent_Turnouts[Turnout][direzione]:
#Se nel sensore c'è qualcosa, attiva lo scambio, almeno che non sia passivo,
#in quel caso deve fermare il treno e poi farlo ripartire quando passa
____________________________________________

            if data.Sensors["Sensore {}".format(item)][direzione+1] != "" :
                #-1 vuoldire cambio passivo
                if natural_link == -1:
                    print("STOP")

                #Assegno il percorso che è occupato.
                data.root_occupied = item
                return natural_link==data.root_occupied

        return False
```

8.4 Testing and Optimization

After initial implementation, the automation system **underwent rigorous testing** to verify its **operation and optimize performance.**
However, being in the development phase, it may still present some **critical issues**.

8.4.1 Testing Procedures

Testing procedures include:

- **Functional Tests:**
  Verifying the **correct operation** of all system features, including sensor reading, turnout control, and locomotive management.

8.4.2 Optimization

Optimization consists of:

- **Performance Improvement:**
  Optimization of the control algorithm and Arduino UNO sketches to ensure a **rapid, lag-free response**.
- **Problem Solving:**
  Correction of any **bugs** encountered during testing and **improvement of overall system stability**.

8.5 Final Considerations

Despite the numerous challenges faced during **development and the current optimization phase**, the system is **designed to be robust, efficient, and user-friendly**, with ample possibilities for **future extension and improvement.**

9 Future Developments

This section outlines the future ideas intended for implementation **following the completion of the Automation System**.

9.1 Further Improvements

Other improvements considered for future development include:

- **On-board Camera Installation for Each Locomotive**
  This improvement has been conceived and partially integrated into the test version of the application.
  Installing cameras on each locomotive and managing the video feeds directly from the application is one of the **key objectives to be achieved**.

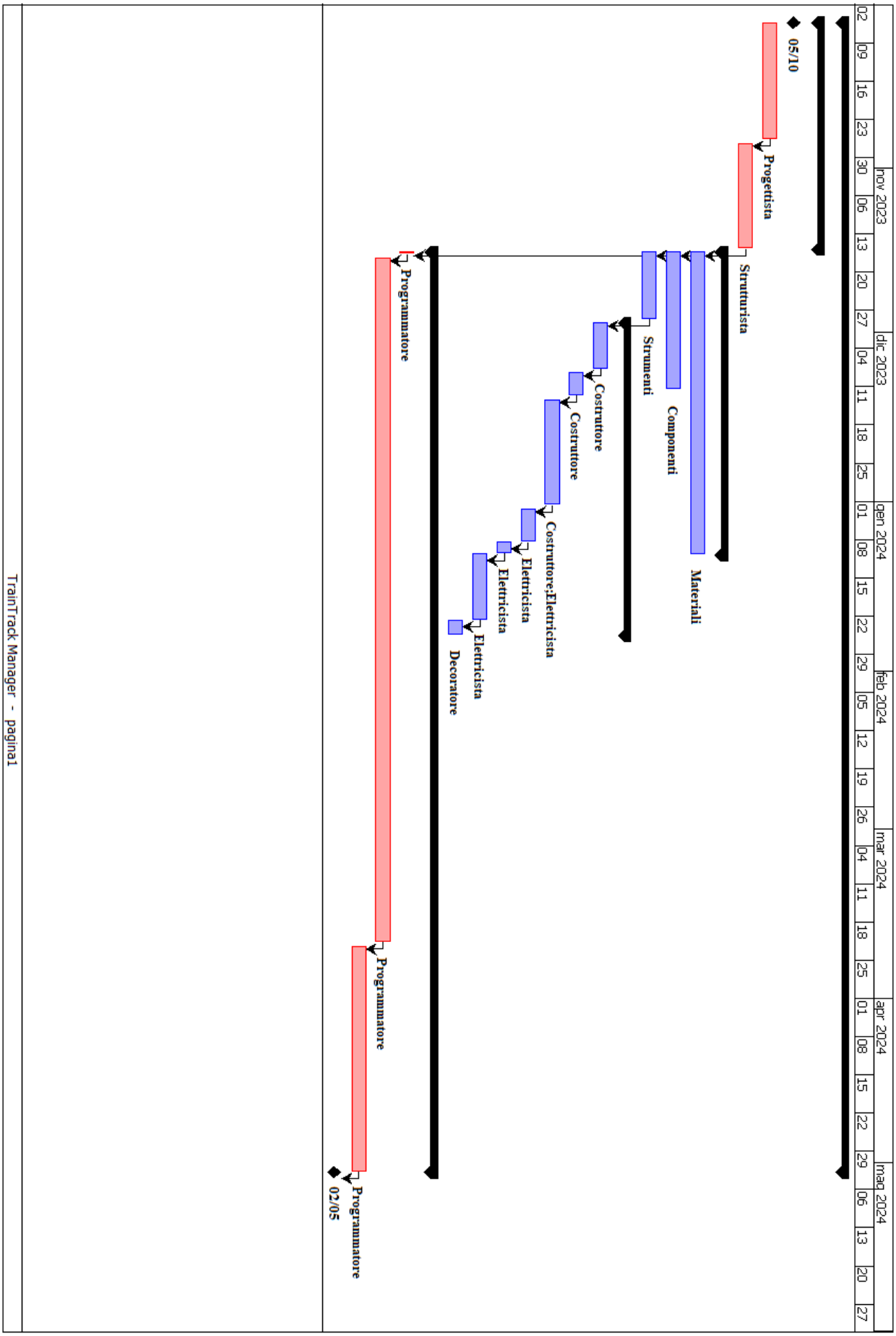
- **Remote Control**
  This option aims to replace the current Arduinos with **Wi-Fi enabled** versions, such as the Arduino R4 Wi-Fi, or alternatively using **ESP-32 modules** (provided performance remains stable). This would allow **starting and controlling the system remotely**, potentially through a purpose-built web or mobile interface.

- **Scalability of Layout Construction in the GUI**
  Another planned project is to expand user choice by providing the ability to **create and edit custom track layouts** and **save them for future use**. This modification would allow the **application to be used for any track design.**

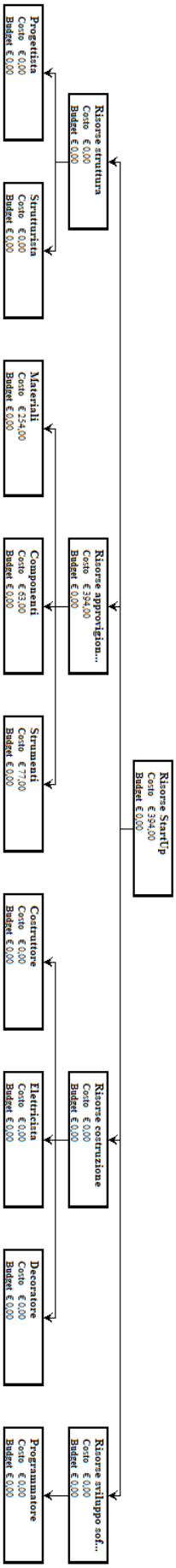
10 Gantt Diagram

		WBS	Nome	Durata	Data di Avvio	Data di chiusura	Predecessori	Nome risorsa
1		**1**	**StartUp Circuito Locomotive**	**151 giorni**	**05/10/23 8.00**	**02/05/24 17.00**		
2		**1.1**	**Struttura**	**30 giorni**	**05/10/23 8.00**	**15/11/23 17.00**		
3		1.1.1	Inizio	0 giorni	05/10/23 8.00	05/10/23 8.00		
4		1.1.2	Progettazione generale	16 giorni	05/10/23 8.00	26/10/23 17.00		Progettista
5		1.1.3	Ricerca informazioni sui materiali da utilizzare	14 giorni	27/10/23 8.00	15/11/23 17.00	4	Strutturista
6		**1.2**	**Approvigionamento**	**40 giorni**	**16/11/23 8.00**	**10/01/24 17.00**		
7		1.2.1	Materiali	40 giorni	16/11/23 8.00	10/01/24 17.00	5	Materiali
8		1.2.2	Componenti	17,5 giorni	16/11/23 8.00	11/12/23 13.00	5	Componenti
9		1.2.3	Strumenti	9 giorni	16/11/23 8.00	28/11/23 17.00	5	Strumenti
10		**1.3**	**Costruzione**	**41,2 giorni**	**29/11/23 8.00**	**25/01/24 9.36**		
11		1.3.1	Costruire il tavolo	7 giorni	29/11/23 8.00	07/12/23 17.00	9	Costruttore
12		1.3.2	Predisporre il tavolo	3 giorni	08/12/23 8.00	12/12/23 17.00	11	Costruttore
13		1.3.3	Creare il circuito	14 giorni	13/12/23 8.00	01/01/24 17.00	12	Costruttore;Elettricista
14		1.3.4	Installare componenti	4,5 giorni	02/01/24 8.00	08/01/24 13.00	13	Elettricista
15		1.3.5	Comporre la centralina	2 giorni	08/01/24 13.00	10/01/24 13.00	14	Elettricista
16		1.3.6	Test componenti elettrici	8,2 giorni	10/01/24 13.00	22/01/24 14.36	15	Elettricista
17		1.3.7	Decorare	2,5 giorni	22/01/24 14.36	25/01/24 9.36	16	Decoratore
18		**1.4**	**Sviluppo software**	**121 giorni**	**16/11/23 8.00**	**02/05/24 17.00**		
19		1.4.1	Creazione dell'ambiente di sviluppo	1 giorno	16/11/23 8.00	16/11/23 17.00	5	Programmatore
20		1.4.2	Sviluppo codice	90 giorni	17/11/23 8.00	21/03/24 17.00	19	Programmatore
21		1.4.3	Testing e debugging	30 giorni	22/03/24 8.00	02/05/24 17.00	20	Programmatore
22		1.4.4	Fine	0 giorni	02/05/24 17.00	02/05/24 17.00	21	

TrainTrack Manager


nov 2023
dic 2023
gen 2024
feb 2024
mar 2024
apr 2024
mag 2024
02 09 16 23 30 06 13 20 27 04 11 18 25 01 08 15 22 29 05 12 19 26 04 11 18 25 01 08 15 22 29 06 13 20 27
05/10
Progettista
Strutturista
Materiali
Componenti
Strumenti
Costruttore
Costruttore
Costruttore;Elettricista
Elettricista
Elettricista
Elettricista
Decoratore
Programmatore
Programmatore
Programmatore
02/05
TrainTrack Manager - pagina1

11 WBS


Risorse StartUp
Costo € 394,00
Budget € 0,00
Risorse struttura
Costo € 0,00
Budget € 0,00
Progettista
Costo € 0,00
Budget € 0,00
Strutturista
Costo € 0,00
Budget € 0,00
Risorse approvigion...
Costo € 394,00
Budget € 0,00
Materiali
Costo € 294,00
Budget € 0,00
Componenti
Costo € 65,00
Budget € 0,00
Strumenti
Costo € 17,00
Budget € 0,00
Risorse costruzione
Costo € 0,00
Budget € 0,00
Costruttore
Costo € 0,00
Budget € 0,00
Elettricista
Costo € 0,00
Budget € 0,00
Decoratore
Costo € 0,00
Budget € 0,00
Risorse sviluppo sof...
Costo € 0,00
Budget € 0,00
Programmatore
Costo € 0,00
Budget € 0,00

12 Cost Tables

These tables list the **costs related to the project since its start date**; any materials not mentioned in these tables were already in my possession.

Table 1: Cost table for Amazon orders

Type	**Description**	**Quantity**	**Date**	**Price(€)**
Component	decoder	3	Oct 11, 2023	**45**
Material	turnouts	3	Oct 14, 2023	**57**
Component	optocouplers	50	Nov 18, 2023	**8**
Tool	Lianshi 18V 5A 90W Power Supply	1	Dec 19, 2023	**22**
Material	Electrical Wire, Awg 30 15m	1	Jan 1, 2024	**16**
Material	Single Row Female PCB Header	1	Jan 5, 2024	**12**
Material	Terminal Block Connector	1	Jan 5, 2024	**13**
Total Amazon	Total sum			**173**

Table 2: Cost table for eBay orders

Type	**Description**	**Quantity**	**Date**	**Price(€)**
Component	DCC locomotive socket	3	Oct 13, 2023	**10**
Material	turnouts	7	Oct 15, 2023	**33**
Material	12V relays	10	Nov 19, 2023	**45**
Total Ebay	Total sum			**88**

Table 3: Cost table for Arduino.cc orders

Type	**Description**	**Quantity**	**Date**	**Price(€)**
Tool	motor shield rev 3	1	Oct 21, 2023	**30**
Tool	Arduino Mega 2560	1	Nov 15, 2023	**25**
Total Arduino.cc	Total sum			**55**

Table 4: Cost table for TEMU orders

Type	**Description**	**Quantity**	**Date**	**Price(€)**
Material	order Dec 24	1	Dec 24, 2023	**23**
Material	order Dec 28	1	Dec 28, 2023	**8**
Material	order Jan 2	1	Jan 2, 2024	**8**
Material	order Jan 21	1	Jan 21, 2024	**17**
Material	order Jan 22	1	Jan 22, 2024	**22**
Total TEMU	Total sum			**78**

Table 5: Summary of all project costs

Grand Total	Sum of all previous tables	**394 €**

References

[1] "Electrical Circuits Technical Documentation." [Online]. Available: https://docs.google.com/document/d/1AeFnw-B1Prgavqx18H5ri2PBlTm2iWFwRN4Kjna7XKo/edit?usp=sharing

[2] "Arduino DCCpp Library." [Online]. Available: https://github.com/DccPlusPlus/BaseStation

[3] "Arduino RFID Library for MFRC522." [Online]. Available: https://github.com/miguelbalboa/rfid

[4] "TrainTrack Manager." [Online]. Available: https://github.com/alexein1001/translating

[5] "GUI Technical Documentation." [Online]. Available: https://www.traintrackdocs.altervista.org/

QR_CODES

Technical Documentation for the circuits

DCCpp Library

MFRC522 Library

TrainTrack Manager GitHub Repository

TrainTrack Manager Technical Documentation